Reflective Journal: Lab 06 Regression and Classification

Name: Joseph Clay
Course: ITAI 1371
Date: October 14, 2026

1. Where I started

I went into this lab thinking that once I finished the notebook, I'd have a full, comprehensive understanding of how these models work and whether they actually predict well. What I really got was the foundation: how to tell the two types of problems apart, how to apply each one, and how to evaluate them to some extent.

2. What clicked

The first thing that clicked was that the question decides the model. If the answer is a bucket, something like yes or no or survived or died, then we're classifying. If the answer is on a spectrum, a number with continuity like how much someone paid, then it's regression and we use linear regression.

The second thing was why we don't use a straight line for a yes-or-no question. A linear model has no max or min. Its line keeps going toward positive and negative infinity, so it can spit out results like −0.3 or 1.4 that make no sense as an answer. Logistic regression fixes that with the S-curve (the sigmoid). It takes the same kind of calculation and squashes it into a range we can actually work with, 0% to 100%, which reads as a probability. That's also why "logistic regression" is a classifier even though the name says regression.

3. Challenges and how I worked through them

Walking through Task 1 step by step is where I really learned the workflow:

- **Create the model.** `lr_model = LinearRegression()` builds an actual model from the class, like building from a blueprint.
- **Train it.** `.fit()` needs two arguments, the training features and the training answers (`X_train_reg`, `y_train_reg`).
- **Predict.** `.predict()` only takes features (X), never the answers (y). The model had already seen the training features, and the goal is to see how it does on data it hasn't seen, so I fed it the test features.
- **Evaluate.** When I first saw MSE, my reaction was basically "what is this?" Breaking down the name helped: for each passenger, take the difference between the predicted fare and the actual fare (the error), square it so negatives don't cancel out positives, then average all of those into one number. I passed it the actual fares first and the predictions second, `mean_squared_error(y_test_reg, y_pred_reg)`, which is the order scikit-learn expects.

My MSE came out to 3364.92, which wasn't usable because it's in squared units of money. Taking the square root (RMSE) put it back into money: 58.01. To do that I needed `np.sqrt`, and if NumPy were never imported, Python would throw a NameError. So I added the import myself. My notebook prints the result as dollars, but I later found out that the Titanic fares are in 1912 British pounds, so the real number is £58.01.

Task 2 followed the same blueprint idea, this time using the `LogisticRegression` class that was imported. I created the model with `log_model = LogisticRegression()`, trained it on the classification training data with `.fit()`, and predicted survival on the test set with `.predict()`. Because the answer here is a bucket (survived or didn't), I scored it with `accuracy_score`, which just counts how many predictions were right. It came out to 74.83%. Seeing the same create, fit, predict, evaluate steps work for a completely different kind of question is what made the workflow stick.

4. Connections

What connected for me is that linear and logistic regression aren't two totally separate ideas. Logistic regression takes the same kind of weighted calculation as the linear model and runs it through the S-curve. The code pattern was the same for both tasks too: create, fit, predict, evaluate. Only the model and the metric changed, because the question changed.

5. Questions that came up

My biggest question was: even after implementing all of this, we were only given enough data to understand the concepts, not to make an accurate conclusion about whether the model predicted well. So what's the point?

Comparing my results to simple baselines helped me partly answer that:

- **Survival:** about 61% of the test passengers died, so always guessing "died" would get 87 of 143 right. My model got 107 right (74.83%). It learned something real from age, class, and sex.
- **Fare:** always guessing the average fare gives an RMSE of about £65. My model got £58, so it was only slightly better. A typical prediction was off by about £24. A few tickets over £500 pushed the RMSE up because squaring makes big misses count much more.

So I can say the survival model learned real patterns and the fare model barely did. What I can't say yet is why the models made the mistakes they did, or whether they're actually good. Age and class weren't wrong features for fare. They just weren't enough to explain it.

6. How my thinking changed

The point of this lab was the foundation, not the final verdict. Each module so far gives me one piece, and later modules (like model evaluation) will change how I judge these same results. Now, before I pick a model, I'll start by naming the question and the target: bucket or spectrum. When I get a score, I won't trust it until I compare it to a simple baseline. And I understand now that a metric tells me how much my features can explain, not just whether my code ran.